Parul® University
Vadodara, Gujarat

NAAC GRADE A++

Information and Communication Technology

Object-Oriented Programming (Classes and Objects)

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor(CSE), PIT
Parul University

5.3.1 Access Modifiers

Access modifiers in Java are keywords used to set the visibility (access level) of classes, methods, constructors, and variables. They help enforce encapsulation, one of the core principles of OOP.

🔰 Types of Access Modifiers in Java

Modifier	Class	Package	Subclass	World
public	✅	✅	✅	✅
protected	✅	✅	✅	❌
default (no modifier)	✅	✅	❌	❌
private	✅	❌	❌	❌

🔶 1. public

- Accessible from **anywhere**: same class, other classes, subclasses, even from other packages.
- Ideal for classes, methods, or variables that need **global access**.
- Example

```
public class MyClass {
    public int number = 10;
    public void display() {
        System.out.println("Public method");
    }
}
```

🔶 2. private

- Accessible only within the same class.
- Used to hide data and enforce encapsulation.
- Not accessible by subclasses or other classes, even in the same package.
- Example

```
public class MyClass {
    private int secret = 42;

    private void showSecret() {
        System.out.println("Private method");
    }
}
```

3. protected

- Accessible within the **same package** and in **subclasses** (even in different packages).
- Commonly used when working with **inheritance**.
- Example

```
public class MyClass {
   protected int age = 25;

   protected void display() {
      System.out.println("Protected method");
   }
}
```

4. Default (no modifier)

- If no access modifier is specified, it is called package-private.
- Accessible only within the same package.
- Example

```
class MyClass {
   int number = 10; // default access

   void display() {
      System.out.println("Default method");
   }
}
```

5.3.2 Garbage Collection and the finalize() Method

What is Garbage Collection in Java?

Garbage Collection (GC) in Java is the **automatic process of identifying and reclaiming memory** that is no longer used or accessible in a program. It helps prevent **memory leaks** and improves **memory management** without requiring manual intervention.

Key Concepts of Garbage Collection

- **Heap Memory**: Where all class instances and arrays are allocated.
- **Unreachable Objects**: Objects that no longer have any reference pointing to them.
- **GC Process**: Identifies such unreachable objects and **removes** them to free memory.
- **Automatic**: Java handles it via the **JVM**, without the need for the programmer to deallocate memory.

How Java Performs Garbage Collection

1. **Reference Checking**: GC looks for objects that have no reference in the program.
2. **Mark and Sweep Algorithm**: Marks active objects and sweeps out the rest.
3. **Compacting**: After removal, it compacts memory to optimize space.
4. **Minor vs Major GC**:
    - **Minor GC**: Cleans **Young Generation** memory (short-lived objects).
    - **Major GC (Full GC)**: Cleans **Old Generation** (long-lived objects).

The finalize() Method

```
protected void finalize() throws Throwable {
   // cleanup code
   super.finalize();
}
```

Purpose:

- It was used to perform **cleanup operations** on objects **before** the object is garbage collected.
- Commonly used to close resources like files or network connections.

Important Notes:

- **Not guaranteed to execute**: The JVM may not call finalize() before program ends.
- **Deprecated since Java 9**: Due to **unreliable behavior**, it's replaced by better mechanisms like **try-with-resources**, AutoCloseable.

Why Use Garbage Collection?

- **Simplifies memory management**.
- **Reduces memory leaks**.
- **Improves performance** in long-running applications.

5.3.3 static Keyword

The static keyword in Java is used to **create class-level members**—i.e., members that **belong to the class itself** rather than to any specific object instance.

🔷 1. Static Variables (Class Variables)

📜 **Definition:**
A variable declared with static is shared among all instances of a class.

🛠 **Syntax:**

```
class Example {
   static int count = 0;  // static variable

   Example() {
      count++;
      System.out.println(count);
   }
}
```

🔍 **Key Features:**

- **Shared by all objects of the class.**
- **Initialized only once, at the time of class loading.**
- **Memory is allocated only once in the Method Area (not in the heap).**

📌 **Use Cases:**

- **To maintain a counter that is shared across all instances.**
- **For constants: static final double PI = 3.14159;**

🔷 2. Static Methods

📜 **Definition:**
A method declared with static can be called without creating an instance of the class.

🛠 **Syntax:**

```
class MathUtil {
   static int square(int x) {
      return x * x;
   }
}
```

📞 **Calling:**

```
int result = MathUtil.square(5); // No object needed
```

🔍 Key Features:

- Cannot use this or super keywords.
- Can **only access static variables or call other static methods**.
- Useful for utility or helper methods (like main() method or Math.pow()).

🔶 Example: Static Variable and Method

```
class Counter {
   static int count = 0;

   static void increment() {
      count++;
   }

   void showCount() {
      System.out.println("Count: " + count);
   }
}
```

🔶 Why Use static?

Purpose	Benefit
Share data across instances	Saves memory, ensures consistency
Access methods without objects	Useful for utility functions and entry points like main
Constants	Makes values globally accessible

5.3.4 final Keyword

The final keyword in Java is used to **restrict modification**. When applied to variables, it makes them **constants**. It can also be used with methods and classes to prevent **overriding** or **inheritance**.

🔷 1. Final Variables

📜 **Definition:**
A variable declared with final cannot be reassigned after it is initialized.

🛠 Syntax:

```
final int MAX_VALUE = 100;
```

🔍 Key Points:

- **Must be initialized once, either at declaration or in a constructor.**
- **Acts like a constant.**
- **Common convention: Final variables are written in UPPERCASE (not mandatory but recommended).**

🧪 Example:

```
class Test {
   final int speedLimit = 90; // constant

   void showLimit() {
      // speedLimit = 100; // ❌ Error: cannot assign a value to final variable
      System.out.println("Speed limit is " + speedLimit);
   }
}
```

🔷 2. Final with Reference Variables

- **A final reference variable means the reference cannot be changed, but the object's internal state can be modified.**
- **Example**

```
final int[] arr = {1, 2, 3};
arr[0] = 10;     // ✅ allowed
// arr = new int[5]; // ❌ Error
```

🔷 3. Final Methods

- **Cannot be overridden by subclasses.**
- **Useful when you want to prevent modification of method behavior.**
- **Example:**

```
class Base {
   final void display() {
      System.out.println("This cannot be overridden");
   }
}
```

🔷 4. Final Classes

- **Cannot be extended by any other class.**
- **Used to create immutable or secure classes, like java.lang.String.**

- **Example:**

```
final class Secure {
   // ...
}

// class Test extends Secure { } // ✕ Error
```

➢ Summary of Key Concepts:

- **Access Modifiers:**
  - **private: Within class only**
  - **default: Within same package**
  - **protected: Package + subclass**
  - **public: Anywhere**
- **Garbage Collection:**
  - **Java auto-reclaims unused memory.**
  - **finalize() was used for cleanup (now deprecated).**
- **static Keyword:**
  - **Belongs to the class, not instance.**
  - **Shared across objects, used for utilities/constants.**
- **final Keyword:**
  - **Variable: value can't change**
  - **Method: can't be overridden**
  - **Class: can't be extended**

➢References:

Oracle Java Documentation (Official Source)

- **Access Modifiers:**

  https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html
- **final, static, and Keywords Overview:**

  https://docs.oracle.com/javase/tutorial/java/nutsandbolts/final.html
- **Garbage Collection:**

  https://docs.oracle.com/javase/specs/jvms/se8/html/jvms-2.html#jvms-2.5.3

GeeksforGeeks

- **Access Modifiers in Java:**

  https://www.geeksforgeeks.org/access-modifiers-java/
- **static Keyword in Java:**

https://www.geeksforgeeks.org/static-keyword-java/

- **final Keyword in Java:**

  https://www.geeksforgeeks.org/final-keyword-java/

- **Garbage Collection in Java:**

  https://www.geeksforgeeks.org/gc-garbage-collection-java/

JavaTpoint

- **Access Modifiers:**

  https://www.javatpoint.com/access-modifiers

- **Static Keyword:**

  https://www.javatpoint.com/static-keyword-in-java

- **Final Keyword:**

  https://www.javatpoint.com/final-keyword

- **Garbage Collection:**

  https://www.javatpoint.com/garbage-collection-in-java

Parul® University
Vadodara, Gujarat

NAAC GRADE A++

https://paruluniversity.ac.in/